Reader Monad

eason: 7 · Advanced FP · Category: Functional · Level: Advanced · Build with Waq

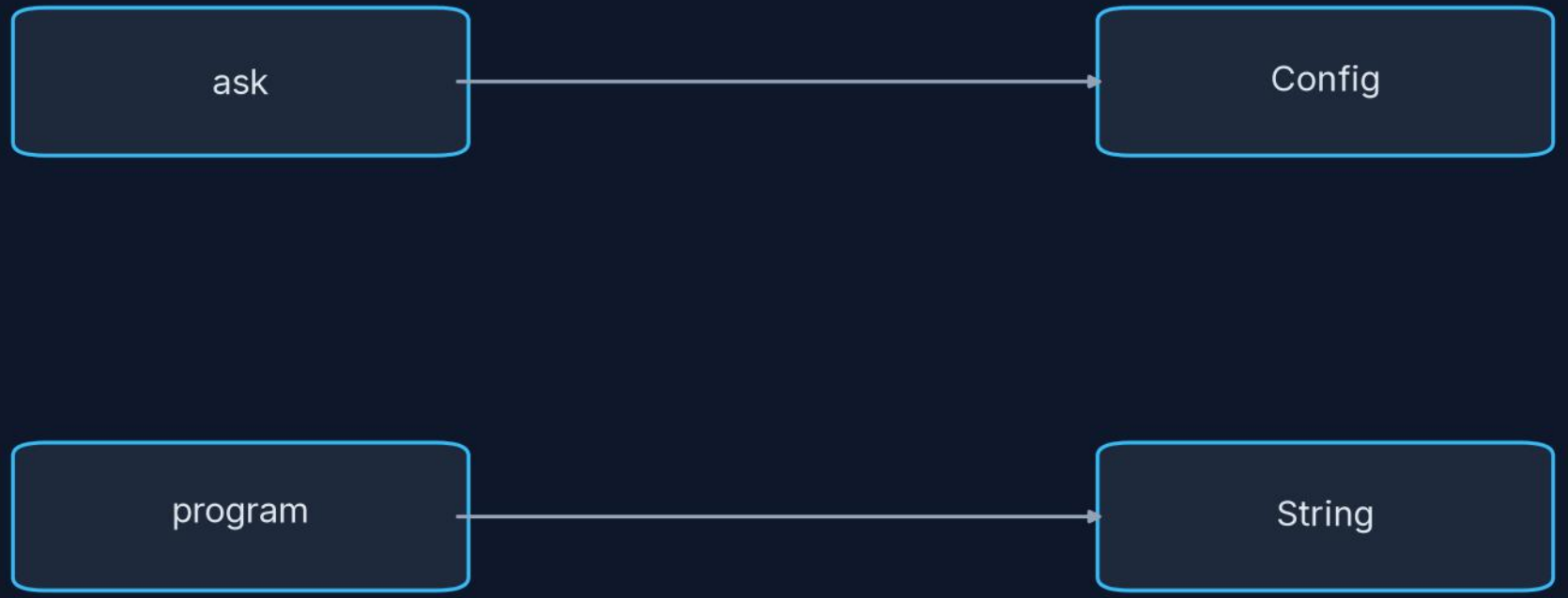
Today's Hook

- Today: Reader Monad as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Reader Monad structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Reader Monad
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/reader/Reader.java
package com.buildwithwaqar.patterns.functional.reader;

import java.util.function.Function;

/** Reader monad — computation that depends on an environment R. */
public record Reader<R, A>(Function<R, A> run) {
    public A run(R env) { return run.apply(env); }

    public <B> Reader<R, B> map(Function<? super A, ? extends B> f) {
        return new Reader<>(env -> f.apply(run(env)));
    }

    public <B> Reader<R, B> flatMap(Function<? super A, Reader<R, B>> f) {
        return new Reader<>(env -> f.apply(run(env)).run(env));
    }

    public static <R, A> Reader<R, A> pure(A a) { return new Reader<>(env -> a); }
    public static <R> Reader<R, R> ask() { return new Reader<>(Function.identity()); }
}

record Config(String greeting, String name) {}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.reader;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Reader walkthrough ===");
    Reader<Config, String> program = Reader.<Config>ask()
      .flatMap(cfg -> Reader.pure(cfg.greeting() + " " + cfg.name() + "!"));
    System.out.println(program.run(new Config("Namaste", "Waqar")));
  }
}
```

Walkthrough Output

```
=== Reader walkthrough ===
Namaste Waqar!
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Reader Monad
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Reader Monad → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.